

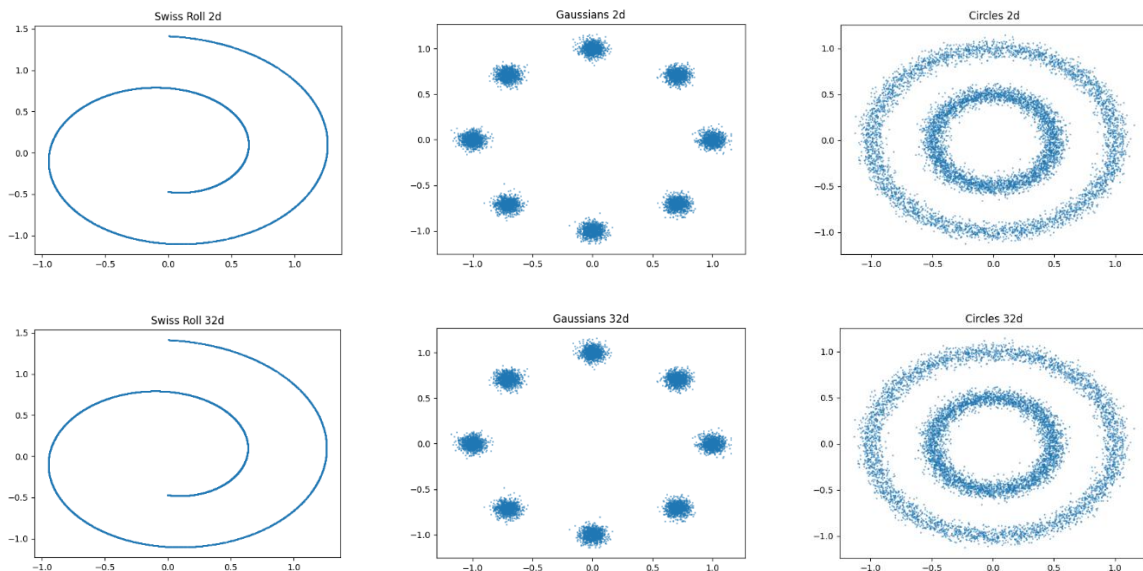
COMP4680 A2 – Flow Matching

Basic Experimental setup

- The diffusion model was implemented exactly as suggested in the assignment description
- The model was trained with the following hyperparameters:
 - Batch size of 1024
 - 25,000 training steps
 - Adam optimizer
 - Cosine Annealing Learning Rate schedule between 10^{-3} and 10^{-4} (this was found to perform better than a fixed 10^{-3})
 - Uniform t sampling between 0 and 1 (per-point sampling; per-batch sampling was found to perform worse due to poor t -coverage)
- The model was sampled using 50-step Euler ODE as suggested in the assignment description

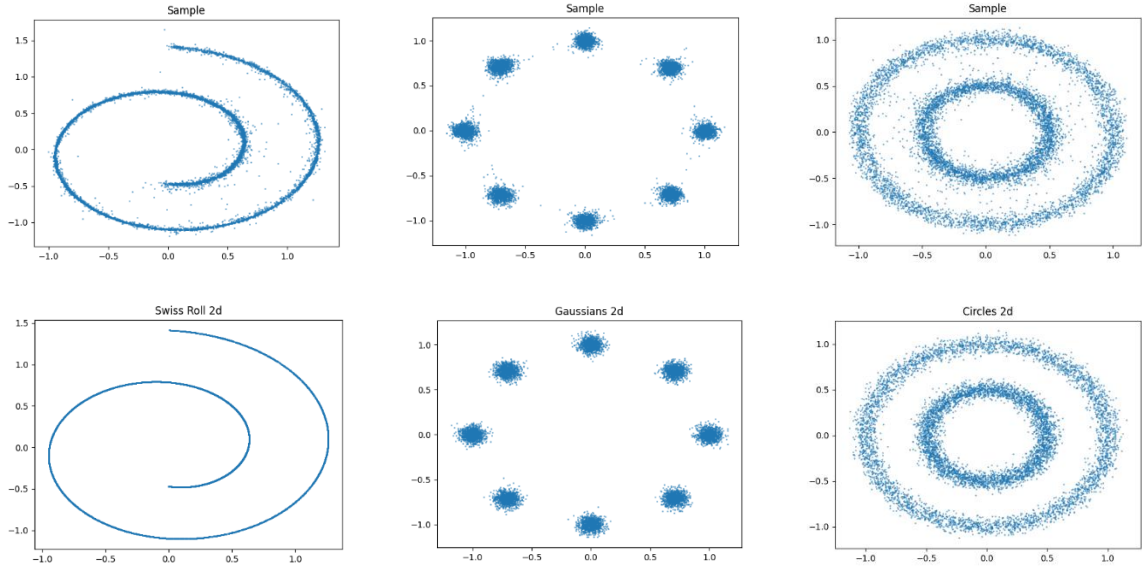
Task 1.1: Data visualization

Below are the three raw datasets, visualized directly from 2d embedding (top row) and projected from 32d embedding (bottom row). This was done using the provided visualization functionality.



Task 1.2: v-prediction flow matching

The following datasets were sampled in 2d (above) using the default training implementation described above, with ground truth data for comparison (below):



As we observe, the v-prediction flow matching generates a clearly recognizable but slightly noisy version of each of the training datasets, at $D=2$.

Task 2: Flow matching parameterization

The forward process for flow matching is:

$$z_t = (1-t)x + t\epsilon \quad (1)$$

Where v is defined as:

$$v = \epsilon - x \quad (2)$$

If we eliminate ϵ from (1) and (2) we get:

$$z_t = (1-t)x + t(v+x) \quad (3)$$

By rearranging (3), we get the following conversion formulae:

1. If we have v-prediction and x-loss we compute the loss term:

$$\text{Loss}(\theta) = \text{mse}(x, x_\theta) \text{ where } x_\theta = z_t - tv_\theta$$

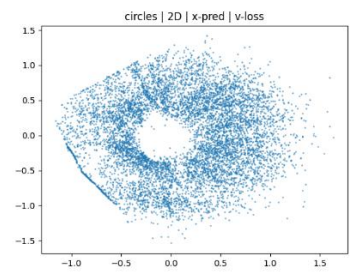
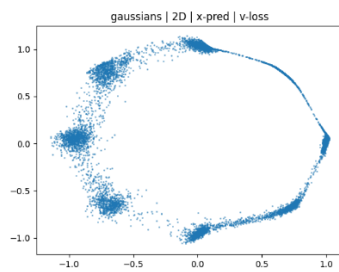
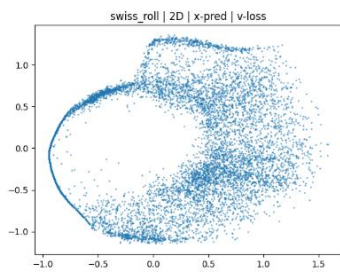
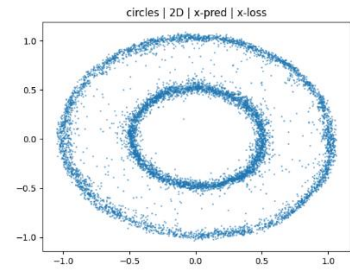
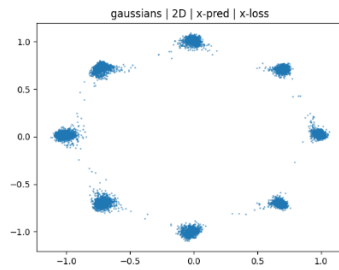
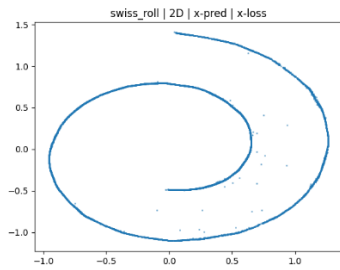
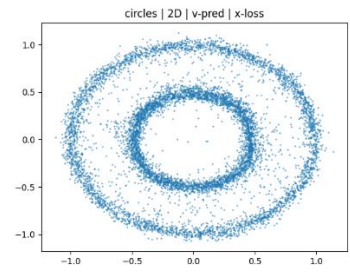
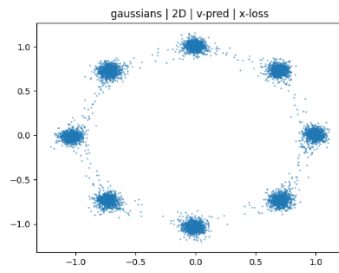
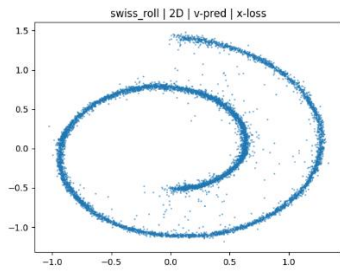
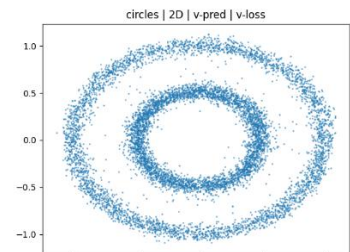
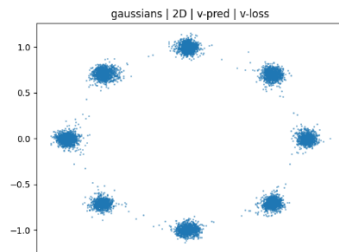
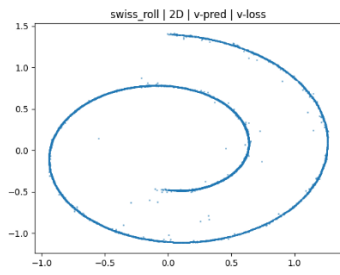
2. And if we have x-prediction with v-loss we compute the loss term:

$$\text{Loss}(\theta) = \text{mse}(\epsilon - x, v_\theta) \text{ where } v_\theta = (z_t - x_\theta)/t$$

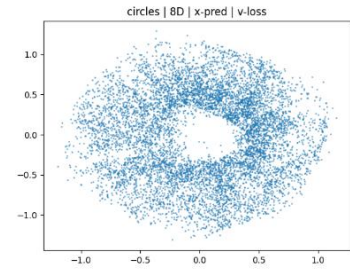
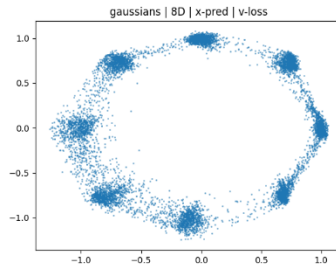
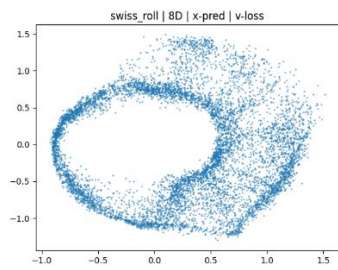
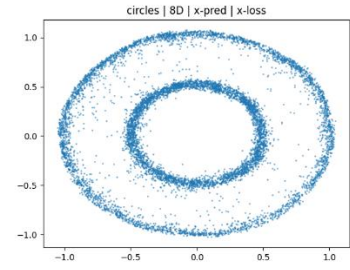
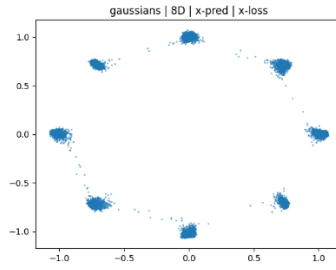
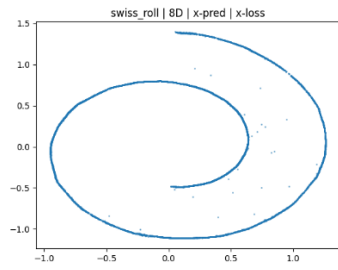
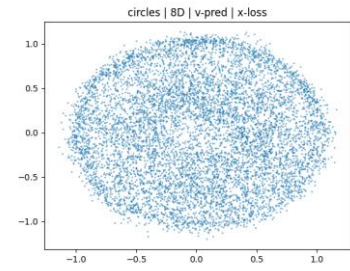
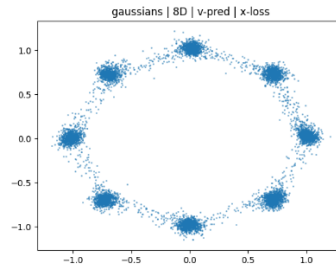
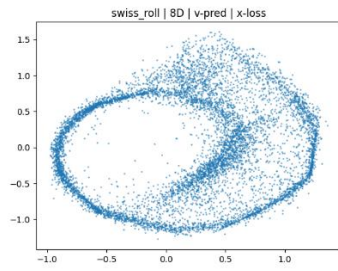
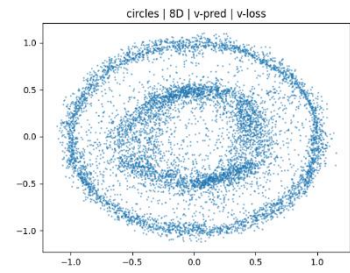
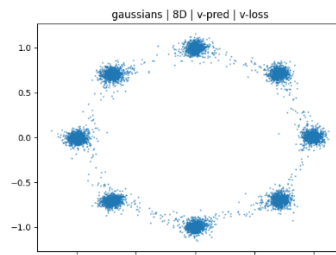
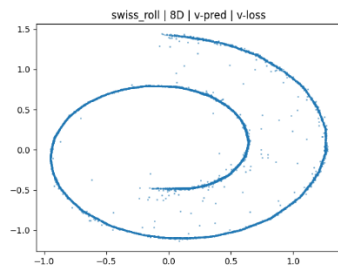
Task 2.2: Comparing parameterizations across 2d, 8d, 32d

Flow matching was implemented with the four different prediction/loss configurations for each dataset in 2d, 8d and 32d. The only change to the default training configuration was clamping t at 10^{-4} , to avoid dividing by $t \approx 0$ as this would cause numerical instability in the loss values and gradients. The sampled results for each dataset, prediction quantity, loss quantity and dimension are shown on the following page.

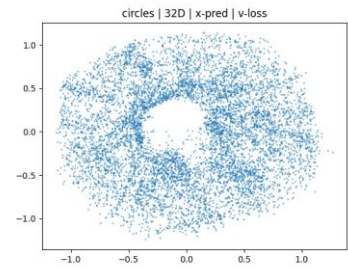
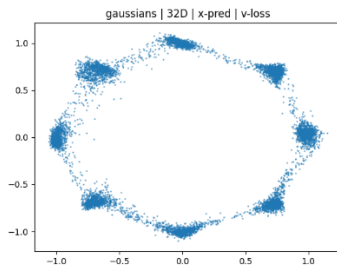
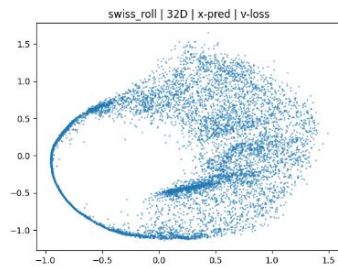
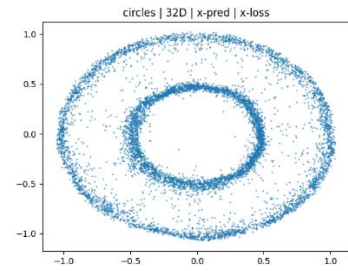
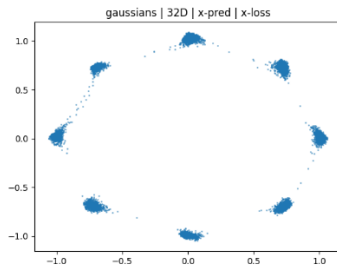
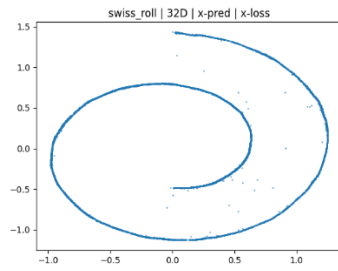
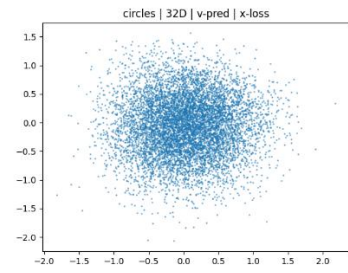
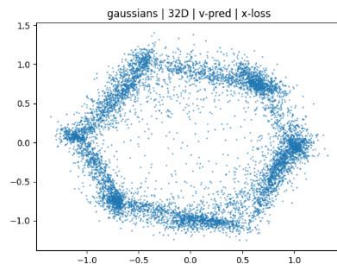
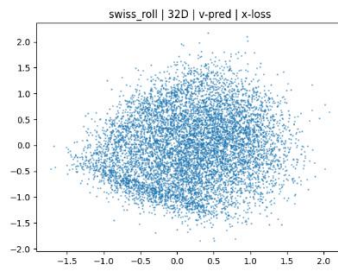
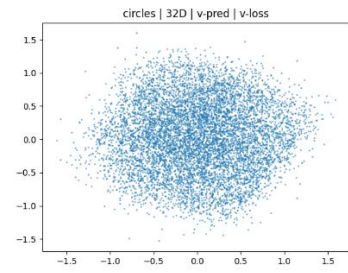
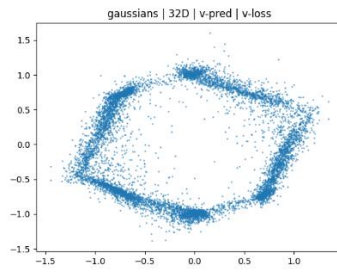
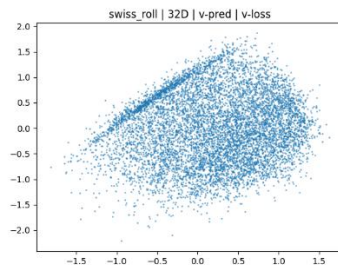
2d results:



8d results:



32d results:



Questions:

1. As we observe, x-prediction scales more successfully to higher dimensions than v-prediction. The v-prediction generates good samples at $D=2$ with both loss types, however it generates visibly noisier samples at $D=8$ (particularly on the `circles` dataset and `swiss_roll` dataset with x-loss), and fails altogether at $D=32$, where the sampled datasets are all unrecognizable. Conversely, x-prediction performs just as well at $D=32$ as it does at $D=8$ and $D=2$.
2. Yes, the choice of loss space matters, especially for x-prediction. X-prediction performs significantly better with x-loss than v-loss across all dimensions, because the division by t in the v-loss calculation results in much larger loss gradients for t values close to 0, which creates a loss weighting where the model tries to hyper-optimize over late diffusion steps ($t \approx 0$) at the expense of important earlier diffusion steps. Thus, we observe much noisier generation samples when using v-loss with x-prediction across all dimensions. With v-prediction, the opposite effect applies – the multiplication by t when computing x-loss creates a loss weighting toward early diffusion steps ($t \approx 1$) at the expense of later diffusion steps. Thus, for v-prediction, we observe that the samples generated with x-loss have similar overall structure to those with v-loss but are always a bit noisier.
3. At $D=8$ or $D=32$, the dataset is projected from 2d onto a 2d manifold in a higher dimensional space. If we use the raw data as the prediction target (x-prediction) the rank of the prediction target is 2 (the rank of x), regardless of the ambient dimension. This means that in training, the model learns to only ever predict points on a 2d manifold within D -dimensional space – concretely, all model weights corresponding to the rest of the D -dimensional space will be ≈ 0 after training. Thus, x-prediction scales well to higher dimensions because the rank of the underlying dataset is preserved.

The same does not apply to v-prediction. This is because the loss, ϵ , is sampled from the full D -dimensional space. This means that the prediction target, $v = \epsilon - x$, is not confined to a 2d manifold in D -dimensional space. Indeed, the rank of v is D since this is the rank of ϵ . This means that the model must optimize exponentially more nonzero weights in higher dimensions compared to $D=2$, which is a fundamentally harder problem. This is why x-prediction works at scale, while v-prediction fails.

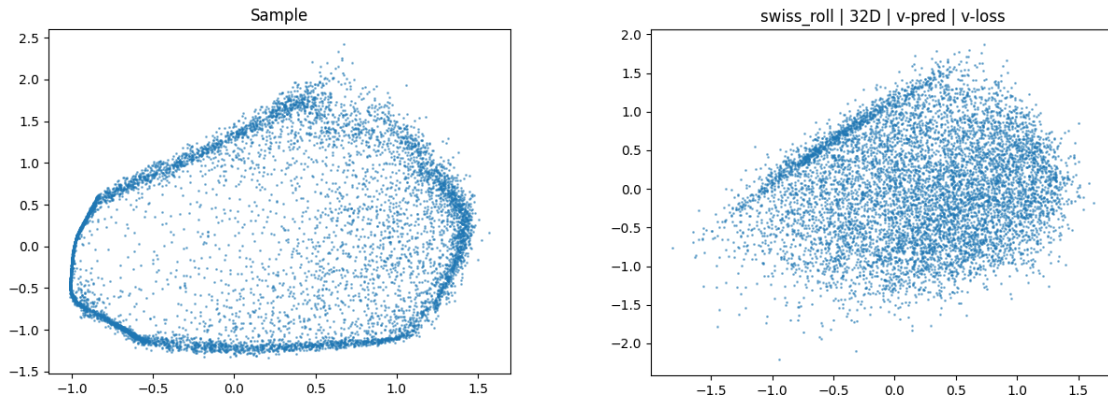
Task 3: Recovering v-prediction at $D=32$

The RAE paper suggests implementing a dimension-dependent noise scheduling to improve v-prediction training at higher dimensions. This was implemented exactly as per the paper with $m=32$ (ambient dimension) and $n=2$ (intrinsic dimension). That is, in the training loop, after sampling t uniformly from $[0, 1]$, t -shift was applied as follows:

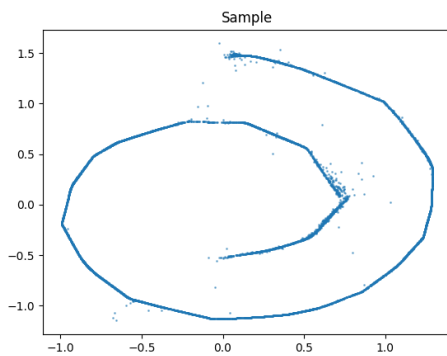
$$t_{\text{shifted}} = \alpha * t / (1 + (\alpha - 1) * t) \quad \text{where } \alpha = \sqrt{m/n}$$

This value was then used to calculate z_t , rather than the sampled t . The reason for this is that when the 2d data is projected into 32d and noise is sampled in 32d, most of the sampled noise falls off the 2d projection manifold, and so the model learns to ignore it. Therefore, in 32d it is better to train the model primarily on more heavily noised samples, so it learns to predict the underlying distribution within the projection manifold.

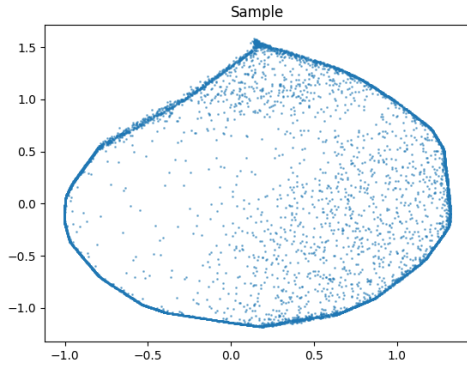
The following shows of a comparison of the default training configuration with t-shifting (left) versus without (right):



T-shifting improved the sampling performance, but it is still a far cry from the performance of x-prediction at $D=32$. In the next experiment, the number of training steps was increased from 25000 to 100000 (with t-shifting implemented), with a Cosine Annealing Learning Rate schedule between $1e-3$ and $1e-5$. The following sample was generated with this configuration:



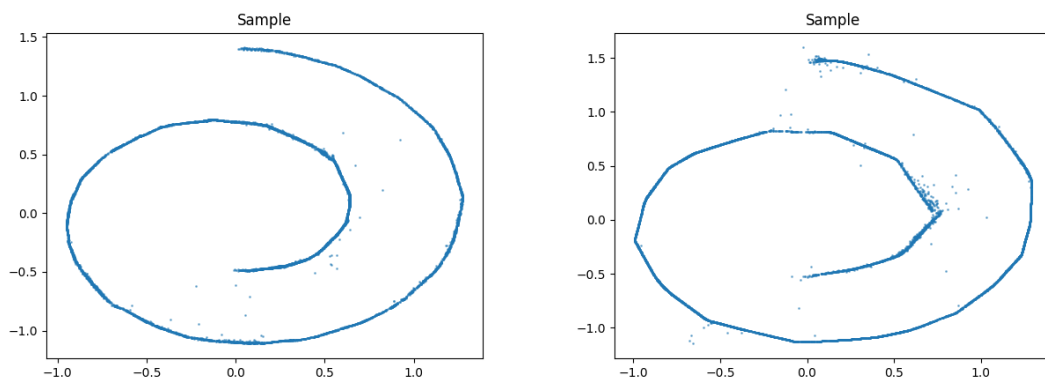
With setup, v-prediction performed comparably to x-prediction (although the generated sample has a few sharp corners which should be smooth curves). As a separate experiment, the learning rate was set lower to a Cosine Annealing schedule between $5e-4$ and $5e-6$. However, this caused sample generation quality to drop noticeably:



Increasing the number of Euler steps in sampling was tried and found to have no noticeable effect on sample quality.

Questions:

1. V-prediction's failure is not fundamental with this setup, since the model width of 256 exceeds the embedding dimension of 32. As demonstrated above, by implementing t-shifting and increasing number of training steps, v-prediction was able to produce a reasonable quality generation sample at $D=32$.
2. I tried implementing t-shifting, increasing the number of training steps, decreasing the learning rate and increasing the number of Euler steps in sampling. My approach which achieves good sample quality is significantly more computationally expensive than the default setup with x-prediction, as it takes 4 times as many training steps, and also calls the (inexpensive) t-shifting function every training step.
3. Below is a comparison of the generated samples using x-prediction (left) vs v-prediction (right) with t-shifting and 10000 training steps:

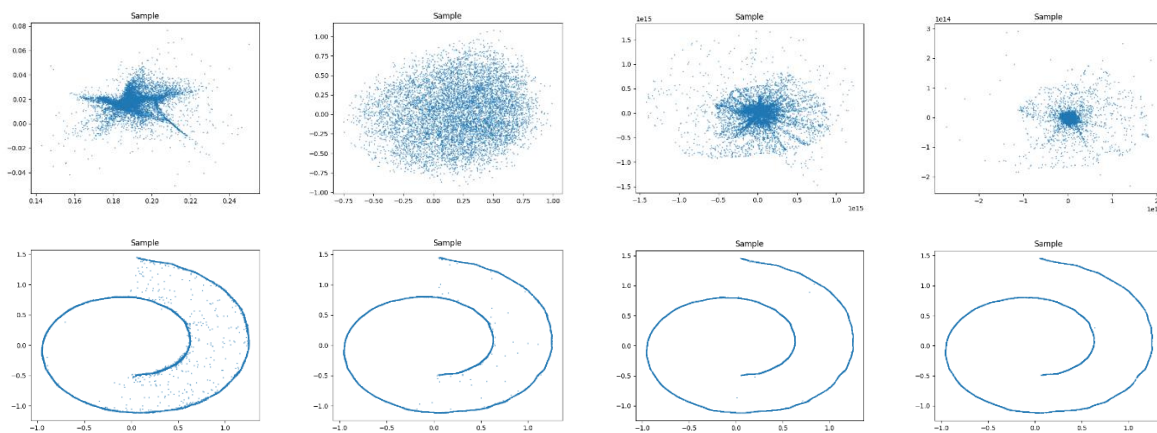


As we can see, applying t-shifting and increasing the training steps did not improve performance with x-prediction like it does with v-prediction. This is because x-prediction only ever learns points on a 2d manifold in 32d x-space, whereas v-prediction is not confined to a subspace of 32d v-space, and thus v-prediction requires longer training. T-shifting is also ineffective for x-prediction since the prediction targets (x) are independent of noise.

4. V-prediction works in practice for image generation models such as Stable Diffusion 3 and FLUX, because real image data usually has an intrinsic dimension close to its ambient dimension, unlike our toy case where we have an intrinsic dimension of 2 and ambient dimension of 32. This means the problem we have of most of the noise falling off manifold doesn't apply in real image generation. Furthermore, we demonstrated that it is still possible to recover v-prediction by optimizing the training configuration, as would be done in practice. Thus with appropriate model size and training configuration, v-prediction is successful in real image generation models.

Task 4.1: Sampling efficiency

Using the default training setup with x-prediction and x-loss on the `swiss_roll` dataset ($D=32$), the following samples were generated using 1, 2, 5, 10, 20, 50, 100 and 200 Euler steps:



As we can see, sampling fails catastrophically with fewer than 20 steps but performs very well with more than 20 steps. The greater the number of Euler steps, the cleaner the sample generated, to the point where at 200 steps there is only one single visible point not on the spiral.

Task 4.2: Mean flow

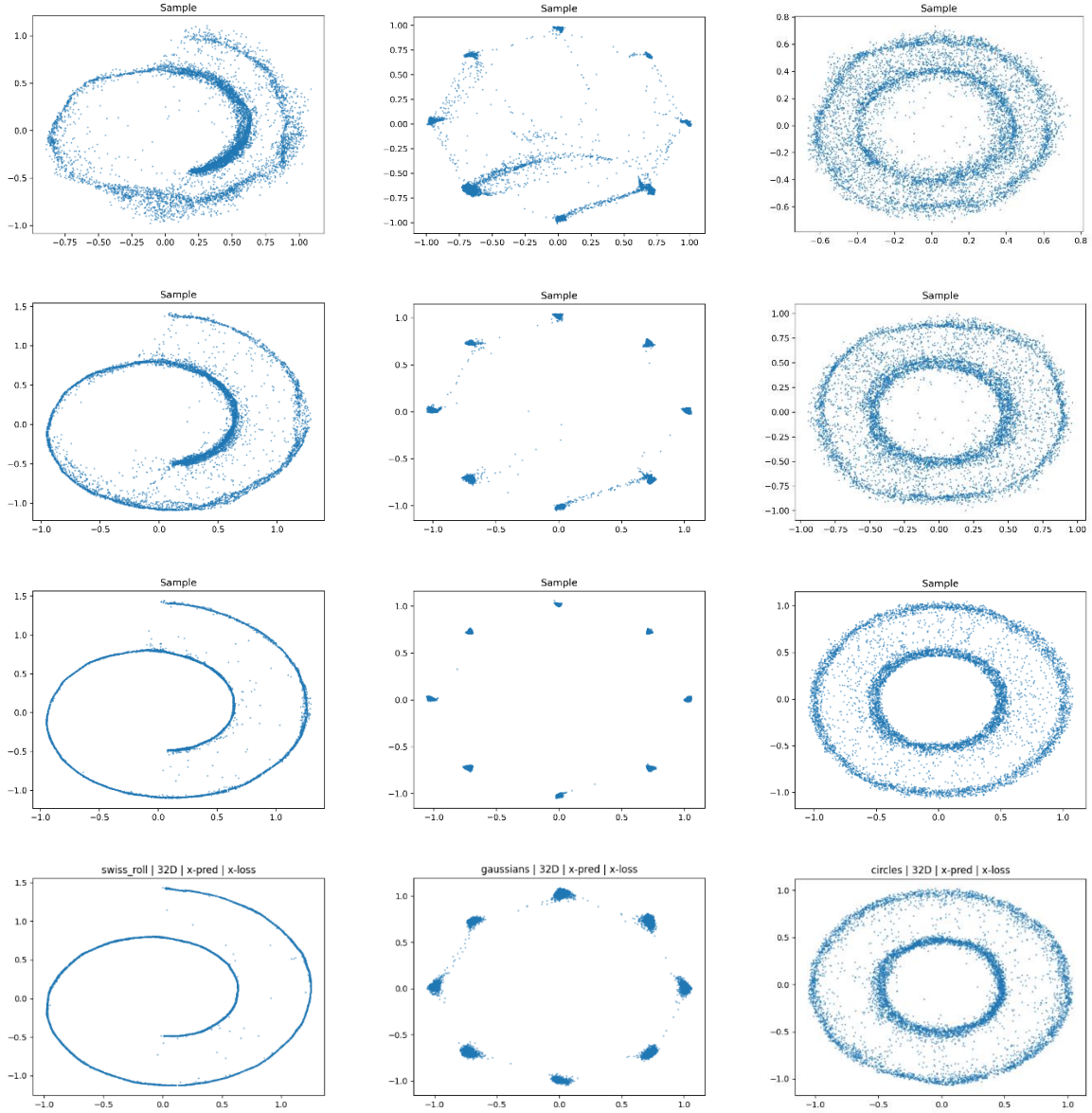
Mean flow was implemented with x-prediction, because x-prediction scaled better to 32d than v-prediction in part 2. After some experimentation, the following training configuration was used as it was found to perform the best:

- A 25% flow matching ratio (rather than the suggested 50%)
- Gradient clipping to improve training stability
- Loss weighting, implemented as per section 4.3 (“Loss metrics”) in the mean flow paper (with $p = 1.0$ as this was reported to be the best performing choice).

That is, given mean flow prediction error err , the model was passed the weighted loss:

$$L = \text{mean}(w * err^2) \quad \text{where } w = 1/(\text{mean}(err^2) + 10^{-3})$$

The mean flow model was trained on all 3 datasets at $D=32$, and the following samples were generated with 1-step (top), 2-step (middle top) and 5-step (middle bottom) mean flow sampling, with flow matching with x-prediction and x-loss for comparison (bottom):



Questions

1. X-prediction was chosen for mean flow at $D=32$ since it was demonstrated to perform better at higher dimensions than v-prediction in part 2.
2. In flow matching, the model learns the instantaneous diffusion velocity at time t , whereas in mean flow the model learns the average diffusion velocity from time t to time 0. This means that a well-trained mean flow model can predict the average diffusion velocity from $t=1$ (pure noise) to $t=0$ (denoised image) and therefore can perform 1-step generation. This does not work for flow matching, since instantaneous velocity is always changing throughout the diffusion process, and therefore accurate generation requires many small Euler steps (with instantaneous velocity updates after each).
3. Mean flow is more expensive to train than flow matching on account of the Jacobian Vector Product (jvp) operation used to calculate mean velocity (as opposed to the trivial instantaneous velocity calculation $v = \epsilon - x$). However, as discussed in Appendix B.4 of the mean flow paper, the extra overhead of the Jacobian Vector product in this case is actually not massive; the paper reports a 16% time/iteration difference between mean flow and flow matching, and attributes this to the fact that the computed result is subject to stop_gradient, meaning that the additional backward propagation is only applied to the model inputs z_t , t and h ; not the model parameters. Therefore, in our case we can expect that mean flow will be about 16% more computationally expensive to train than flow matching.
4. Compared to the ground truth distributions, all the 1-step generated samples are recognizable but very noisy. This is not really surprising, since with 1-step generation, each point only does one forward pass through the model, and so any slight errors the model makes have no chance to be corrected. Whereas with 2-step and especially 5-step generation, there is more opportunity for forward pass errors to be corrected by subsequent steps. With 5-step generation, the `swiss_roll` and `circles` samples closely resemble the underlying distributions (albeit with some noise), but the `gaussians` sample has a noticeable difference: the gaussian blobs are smaller in size than the underlying distribution. This is because when the model sees a noised point z_t , it computes a weighted average velocity over all data points x which could have been noised to create z_t . Because the `gaussians` distribution is a collection of isolated gaussian blobs, any point z_t which is sufficiently close to one particular blob will be mapped by the model to an average velocity pointing close to the centre of the blob, giving rise to the tighter blobs we observe for few step mean flow sampling. This effect applies to all datasets in principle, but in practice it is not observed with the `swiss_roll` and `circles` datasets, since in both cases the points lie on continuous manifolds, not in isolated groups.